theory *Week_4_sol*
imports *Main*
begin

King's College London
Department of Informatics
Dr. Mohammad Abdulaziz

Term 1

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 4

About these solutions. Every Isabelle proof method (e.g. *rule*, *simp*), Isar keyword (e.g. *have*, *also*), attribute (e.g. *OF*, *of*) and method modifier (e.g. *arbitrary:*, *simp add:*) is explained in a short comment, marked —, the first time it appears in the solutions, in sheet order from Week 1. Later uses are not explained again, so if you meet an unfamiliar construct, look for its first use in an earlier solution. Each proof is also labelled with its style; the module's preferred style is declarative (Isar), where every step states what it proves.

Recall the binary trees which we discussed in the lecture and the function *tree_set*.

datatype *'a tree = Leaf | Node 'a tree 'a 'a tree*

fun *tree_set* **where**
tree_set Leaf = {}
| *tree_set* (*Node l a r*) = *insert a* (*tree_set l*) ∪ (*tree_set r*)

Exercise 4.1 Binary Search Trees: Invariant

Recall that a binary tree can be used to make searching for elements easier by imposing the following constraint: the content a of every node *Node l a r* in the binary tree has to be:

1. Strictly greater than the content of all nodes in the left sub-tree l of that node, and
2. Strictly smaller than the content of all nodes in the right sub-tree r of that node.

A tree satisfying such conditions is called a *binary serach tree*, and the above conditions are called the binary search tree invariant.

Write a functional program *bst*, both pen-and-paper and in Isabelle, which can check whether a given tree is a binary search tree.

fun *bst*::*nat tree* ⇒ *bool* **where**

bst Leaf = *True*
| *bst* (*Node l a r*) = ((∀ *x*∈*tree_set l*. *x* < *a*) ∧ *bst l* ∧ (∀ *x*∈*tree_set r*. *a* < *x*) ∧ *bst r*)

Exercise 4.2 Binary Search Trees: Search

If a binary tree satisfies the constraints of a binary search tree, then it can make searching for an element easier. In particular, you can avoid searching in one of the subtrees based on how the value you are looking for compares to the root:

- if it bigger then you only search in the right sub-tree,
- if it is smaller, then you only search in the left sub-tree, and
- if it is the same as the root then you know the element you look for is in the tree.

Write a functional program *isin*, both pen-and-paper and in Isabelle, which can check if a a given value is present in a given binary search tree. The program should not search in a sub-tree where you know the element will not be present.

fun *isin* :: ('*a*::*linorder*) *tree* ⇒ '*a* ⇒ *bool* **where**

isin Leaf x = *False* |
isin (*Node l a r*) *x* =
  (*if x* < *a then isin l x else*
   if x > *a then isin r x*
   else True)

Prove that this program is correct.

lemma *tree_set_isin*: *bst t* ⟹ *isin t x* = (*x* ∈ *tree_set t*)

proof(*induction t*)
  — Style: declarative (Isar); structural induction on trees (new); each case by *auto*.
  case *Leaf*
  then show *?case*
    by *auto*
next
  case (*Node l a r*)
  then show *?case*
    by *auto*
qed

Exercise 4.3 Binary Search Trees: Inserting an Element

Write a functional program that inserts an element into a binary search tree. Make sure that the new tree computed by the program is also a binary search tree.

fun *ins* :: *'a::linorder* ⇒ *'a tree* ⇒ *'a tree* **where**

ins x Leaf = Node Leaf x Leaf |
ins x (Node l a r) =
(if x < a then Node (ins x l) a r else
if x > a then Node l a (ins x r)
else Node l a r)

Prove that your program is correct by showing that the inserted element is actually in the resulting tree, and that the resulting tree is a binary search tree.

lemma *tree_set_ins*: *tree_set (ins x t) = {x} ∪ tree_set t*

proof(*induction t*)
— Style: as above.
case *Leaf*
then show *?case*
by *auto*
next
case (*Node l a r*)
then show *?case*
by *auto*
qed

lemma *bst_ins*: *bst t ⟹ bst (ins x t)*

proof(*induction t*)
— Style: as above, giving *auto* a previously proved lemma (*simp*:).
case *Leaf*
then show *?case*
by (*auto simp*: *tree_set_ins*)
— *auto simp*: *th* lets *auto* also use the lemma *th* as a simplification rule.
next
case (*Node l a r*)
then show *?case*
by (*auto simp*: *tree_set_ins*)
qed

end